Assignment 3

```
Name : Manas More
Roll no. : 25
Course : MCA
Subject : Data Communications and Computer Networks
```

Programming Tasks

The following tasks require implementing CRC in a language of your choice (C, C++, Java, or Python are recommended). Submit source code along with sample output for each task.

Task 1: CRC Generator (Sender Side)

Write a program that:

1. Accepts a binary message string and a generator polynomial (as a bit string) from the user.
2. Appends r zero bits to the message, where r is one less than the length of the generator.
3. Performs modulo-2 (XOR-based) division to compute the CRC remainder.
4. Outputs the remainder and the final transmitted frame (message + CRC).

```python
def bits_count(data):
    bits = 0

    while data > 0:
        bits += 1
        data >>= 1

    return bits


def crc(message, generator):
    msg = int(message, 2)
    gen = int(generator, 2)

    msg_len = bits_count(msg)
    gen_len = bits_count(gen)
```

```
    if msg_len < gen_len:
        print("Error: generator has to be less long than message")
        return ""

    rem = msg << (gen_len - 1)
    rem_len = msg_len + gen_len - 1

    while rem_len >= gen_len:
        rem = rem ^ (gen << (rem_len - gen_len))
        rem_len = bits_count(rem)

    print(f"\nRemainder: {rem:b}\n")

    codeword = ""

    for i in range(msg_len - 1, -1, -1):
        codeword += '1' if (msg & 1) else '0'
        msg >>= 1

    return codeword


def main():
    message = input("Enter message: ")
    generator = input("Enter generator: ")

    codeword = crc(message, generator)

    print(f"Sender codeword: {codeword}")


if __name__ == "__main__":
    main()
```

```
py assignment-3.py
Enter message: 10100101101
Enter generator: 1011

Remainder: 0

Sender codeword: 10110100101
```

Task 2: CRC Checker (Receiver Side)

Write a program that:

1. Accepts a received bit string and the same generator polynomial used at the sender.
2. Divides the received string by the generator polynomial using modulo-2 division.

3. Reports whether the frame is error-free (remainder is all zeros) or corrupted (non-zero remainder).

```
def bits_count(data):
    bits = 0

    while data > 0:
        bits += 1
        data >>= 1

    return bits


def crc_check(codeword, generator):
    cdw = int(codeword, 2)
    gen = int(generator, 2)

    cdw_len = bits_count(cdw)
    gen_len = bits_count(gen)

    if cdw_len < (2 * gen_len):
        print("Error: codeword has to be twice as long as generator")
        return False

    rem = cdw
    rem_len = cdw_len

    while rem_len >= gen_len:
        rem = rem ^ (gen << (rem_len - gen_len))
        rem_len = bits_count(rem)

    return rem == 0


def main():
    codeword = input("Enter codeword: ")
    generator = input("Enter generator: ")

    correct = crc_check(codeword, generator)

    print(f"Received message has {'no ' if correct else ''}error")


if __name__ == "__main__":
    main()
```

Test your program by

- passing an unmodified frame produced by Task 1, and

```
 py assignment-3_2.py
Enter codeword: 10100101101
Enter generator: 1011
```

```
Received message has no error
```

- manually flipping one or more bits before checking.

```
py assignment-3_2.py
Enter codeword: 10101101101
Enter generator: 1011
Received message has error
```